

Multiple Catch Blocks in C# with Examples

In this article, I am going to discuss how to implement **Multiple Catch Blocks in C#** to handle different types of exceptions for a single try block with examples. Please read our previous article before proceeding to this article where we discussed the basics of [Exception Handling in C#](#) with examples. As part of this article, I am going to discuss the following pointers.

1. [How to implement Multiple Catch Blocks in C#?](#)
2. [Is it possible to catch all exceptions using a single catch block?](#)
3. [When should we write multiple catch blocks in C# for a single try block?](#)

How to Implement Multiple Catch Blocks in C#?

It is possible in C#, to write multiple catch blocks for a given try block. When we implement multiple catch blocks in C# for a given try block, then at any given point of time only one catch block is going to be executed and other catch blocks will be ignored. With this kept in mind, let us proceed and see an example of how to implement Multiple Catch Blocks in C#.

Example to Understand Multiple Catch Blocks in C#

Let us see an example and try to understand how to implement multiple catch blocks for a given try block in C# and also try to understand the execution flow. Please have a look at the following example. As you can see, here, we created two catch blocks for the given try block. The first catch block takes the `DivideByZeroException` class as the input parameter and the second catch block takes the `FormatException` class as the input parameter.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result: {Result}");
            }
            catch (DivideByZeroException DBZE)
            {
                Console.WriteLine("Second Number Should Not Be Zero");
            }
            catch (FormatException FE)
            {
            }
        }
    }
}
```

```

        Console.WriteLine("Enter Only Integer Numbers");
    }
    Console.ReadKey();
}
}
}

```

Run the above application and entered values like 10, and 0. It will give you the below output. In this case, when we entered the second number as 0, the CLR implicitly throws the **DivideByZeroException** instance and the catch block which takes as a parameter of DivideByZeroException will handle the exception, and hence the message Second Number Should Not Be Zero will be printed on the console as shown in the below image.

```

Enter First Number
10
Enter Second Number
0
Second Number Should Not Be Zero

```

Again, run the application and entered the values as 10 and abc. This time it will give you the following output. In this case, when we entered the second value as abc, the CLR implicitly throws **FormatException** instance and the catch block which takes as a parameter of FormatException will handle the exception and hence, the message Enter Only Integer Numbers will be printed on the console as shown in the below image.

```

Enter First Number
10
Enter Second Number
abc
Enter Only Integer Numbers

```

Whenever we implement Multiple Catch Blocks in C#, then it is not possible to write the catch blocks in the following manner, it raises to compilation error because the first catch block Exception can handle all the exceptions as it is the parent class of the child exception classes and it does not make any sense to write the further catch blocks as they are never going to be executed.

```

try
{
    //Code
}
catch (Exception ex)
{
    //Code
}
catch (FormatException fex)
{
    //Code
}

```

Is it possible to catch all exceptions using a single catch block in C#?

Yes, it is possible. We can catch all exceptions with a single catch block with the parameter `Exception`. The `Exception` class is the superclass of all Child `Exception` classes and hence it can handle all types of exceptions thrown in the `try` block. We need to use this catch block only for stopping the abnormal termination irrespective of the exceptions thrown from its corresponding `try` block. For a better understanding, please have a look at the following example.

```

using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result: {Result}");
            }
            catch (Exception ex)
            {
                Console.WriteLine("Generic Catch Block...");
            }
            Console.ReadKey();
        }
    }
}

```

Now, in the above example, for any type of exception, the generic catch block is going to be executed. It is always recommended to write a catch block with the `Exception` parameter as the last catch block even

though we are writing multiple catch blocks. It acts as a backup catch block. Following is the syntax to do the same.

```
try
{
    //Code
}
catch(DivideByZeroException dbe)
{
    //Code
}
catch (FormatException fe)
{
    //Code
}
catch (Exception e)
{
    //Code
}
```

In the below example, we are writing the generic catch block as the last catch block and if any exception thrown in the try block is not handled by any specific catch block, then the generic block is going to be executed.

```
using System;
namespace ExceptionHandlingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int Number1, Number2, Result;
            try
            {
                Console.WriteLine("Enter First Number");
                Number1 = int.Parse(Console.ReadLine());
                Console.WriteLine("Enter Second Number");
                Number2 = int.Parse(Console.ReadLine());
                Result = Number1 / Number2;
                Console.WriteLine($"Result: {Result}");
            }

            catch (DivideByZeroException DBZE)
            {
                Console.WriteLine("Second Number Should Not Be Zero");
            }
            catch (FormatException FE)
            {
                Console.WriteLine("Enter Only Integer Numbers");
            }
        }
    }
}
```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine("Generic Catch Block...");
    }
    Console.ReadKey();
}
}
}

```

Now, run the above application and entered big integer values like 23456789234. It will give you the following output. In this case, when we entered the number 23456789234, the CLR implicitly throw the `OverflowException` instance and we don't have any catch block which takes as a parameter of `OverflowException`, hence the generic catch block is going to execute and display the following message.

```

Enter First Number
23456789234
Generic Catch Block...

```

When should we write Multiple Catch Blocks in C# for a Single Try block?

We need to write multiple catch blocks in C# for a single try block because of the following reasons

1. To print messages specific to an exception or
2. To execute some logic specific to an exception

In the next article, I am going to discuss how to create a [Finally Block in C#](#) with Examples. Here, in this article, I try to explain how to implement **Multiple Catch Blocks in C#** with Examples. I hope you understood this as well as enjoy this Multiple Catch Blocks C# with Examples article.